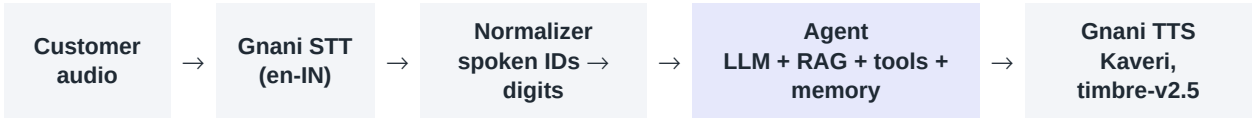
VoiceOps 2026: ZentroMart Voice Support Agent

Lakshya Pandey · From Tokens to Transformers capstone (C&A Club, IIT Guwahati × Gnani.ai) · October 2026
Code: github.com/pandeylakshya207-max/voiceops-zentromart-agent
Run: `python run.py --audio_dir <folder> --out_dir outputs`



One agent serves text and voice. The LLM is pluggable: Groq-hosted **gpt-oss-120b** by default (fallback gpt-oss-20b), and **gemini-3.5-flash-lite** was used for Step 3 and the last Step 2 items when the free Groq quota ran out; retrieval runs locally; speech in and out is Gnani only. Everything a customer is told comes from the 21 knowledge-base documents, the CSV tables or the two provided tools.

Results on the practice set

Metric (52 practice questions, ASR-style text)	Step 1: prompt only	Step 2: first full run	Step 2: final
Fully correct (all checks pass)	**17.3%** (9 / 52)	78.8% (41 / 52)	**98.1%** (51 / 52)
Required facts present (must_include)	26.9%	88.5%	100%
Trap answers avoided (must_not_include)	100%	100%	100%
Correct decision (answer / clarify / escalate / refuse)	76.9%	84.6%	98.1%
Gold evidence retrieved (recall)	n/a (no retrieval)	85.1%	87.9%

Metric (23 practice conversations, 96 customer turns)	Step 3: first clean run	Step 3: after fixes
Turns fully correct	94.8% (91 / 96)	**100%** (96 / 96)
Required facts present	96.9%	100%
Correct decision per turn	96.9%	100%
Expected tool calls made with the right arguments	100%	100%
Gold evidence retrieved (recall)	94.6%	94.6%

Scoring follows the practice files exactly: every `;`-group of `must_include` must appear (alternatives split by `/`), nothing from `must_not_include`, the decision must be acceptable, expected tool calls must match, and evidence recall is measured against `gold_evidence`. Free-tier limits shaped the runs: Steps 1–2 use Groq gpt-oss-120b, falling back to gpt-oss-20b when the daily quota ran out (final Step 2 rows: gpt-oss-120b 25, gpt-oss-20b 21, gemini-3.5-flash-lite 6); the failing items were re-run after each fix (`evaluate.py --redo`). Step 3 ran on gemini-3.5-flash-lite: one clean run of all 23 conversations with the final code, then the 5 conversations with a miss were re-run after fixing the causes listed below. Per-item results are in `results/`.

Step 1: answer with an LLM (prompt only)

A system prompt makes the model ZentroMart's support assistant, sets its limits (no promises, say when unsure, offer a colleague), requires an AI disclosure and asks for JSON with a reply and a decision. Without documents it got **17.3%** right. It never fell into the trap phrases only because it did not know them, and it confidently invented policy: "delivery slots between 9 am and…", 30-day return windows that do not exist, "we only deliver inside India", store pickup "at a nearby outlet" and wrong refund timelines. That is the baseline Step 2 had to beat.

Step 2: ground it in the documents (RAG)

- **Chunking.** Each `kb/*.md` file is split at its `## [SECTION-ID]` headings: one chunk per section, carrying doc_type, status and effective_from from the front matter. Section IDs are what the agent cites and logs.
- **Hybrid retrieval.** BM25 (catches exact IDs like ELE-0012) plus multilingual-e5-small embeddings (catch meaning, Hinglish and Hindi), 0.45 / 0.55 blend, top 6. A small synonym map turns everyday words into policy vocabulary (torn / kharab / toota → damaged, paisa wapas → refund).
- **Precedence.** Policy > agent guide > help > FAQ > marketing is applied as a ranking bonus and stated in the prompt; the precedence section KB-00.1 is always in context; the superseded returns policy is down-weighted and applies only where v4 says so. This is what beats the FAQ-1 trap ("5 to 7 days" vs. instant wallet refunds).
- **Structured facts come from code, not the model.** Product rows (warranty, Assured, price, return category) are looked up automatically when a caller says a SKU or product name; pincode serviceability from pincodes.csv; all date arithmetic from `date_calc` (today = 20 Oct 2026, business days Mon–Sat minus warehouse holidays, return windows in calendar days).
- **"I don't know".** If no section literally states the answer, the agent says it does not have that information and offers a colleague. It may not turn a gap into a policy ("we don't offer X").

- **Traceability.** Every answer logs the retrieved chunk IDs and the IDs/rows it cites (e.g. `REF-2`, `products.csv:ELE-0012`); run.py saves them next to each reply.

Step 3: hold a conversation

- **Call memory.** A call state keeps the order under discussion, the verified order record, failed verification attempts, tickets raised and the caller's language. Follow-ups ("aur refund kitne din mein?") reuse it, and the verified order's payment method and status are added to the retrieval query.
- **Caller verification in code.** When an order ID is heard, `lookup_order(order_id)` runs before the model speaks; when the caller gives 4 or 6 digits, verification runs with them. One retry is allowed, then the order is locked for the call and a human is offered. The expected value is never revealed.
- **Handover only when real.** Approval, compensation, damage and account-security requests pull in the escalation rules (e.g. claims above Rs 10,000 go to Priority Resolution). The agent can never approve anything; it raises `create_ticket` with issue, checks made and reason, and only says a ticket exists when `created=True`.
- **One clear question.** It asks only for what it truly needs; if a date is missing it states the rule instead ("within 30 days of delivery"); requests for another person's data are refused.
- **Security wording in code.** The opening "I need help with my order" and both failed-verification replies are written by code in the caller's language, so they are always correct and cost no LLM call.
- **Labels follow actions.** The decision is derived from what happened: a ticket means escalate, a denied request means refuse (with an apology), a request for the order ID or digits means clarify.

Verifier loop: checks before anything is spoken

Prompt rules alone were not reliable, so code checks each draft reply and sends the model back once, with a specific instruction, when the draft breaks a rule:

Check	Triggered when the draft reply…
Date grounding	states a date no tool, document or the customer gave, or differs from what date_calc returned
Deadline / by-when	answers a 'by when?' or return/claim question without the computed date
Search before 'don't know'	says it has no information without having searched the knowledge base
Needless question	asks for an order ID or detail that the documents plus the caller's words already settle
Catalog facts	leaves out the fact that was asked, e.g. the warranty months of the named SKU
Language	answers in English to a Hinglish or Hindi caller
Damage / security	handles a damage or 'order I never placed' report without checking who must decide

Step 4: give it a voice

- **Ears and voice.** Gnani STT (`en-IN`, which accepts Hindi-English mix) feeds the same Step 3 agent; replies are spoken with Gnani TTS (Kaveri, timbre-v2.5, hi-en for Hinglish). No other speech service is used.
- **Written for the ear.** Replies are 1–3 sentences, no lists or markdown, dates with the year, order numbers read in groups ("8978 2837"), SKUs spelled out, numbers as digits.
- **Spoken input clean-up.** "ele zero zero one to" → ELE-0012; "eight nine seven eight…", "8978 2837", "forty five thirty three" and "four thousand five hundred thirty three" all become digits; Hindi number words too.
- **Recovery from unclear audio.** Empty or filler-only transcripts get "Sorry, I didn't catch that, could you say it again?" in the caller's language; an order number or verification value heard with the wrong number of digits is read back ("I heard 8978 283, which is 7 digits…") without spending an LLM call.
- **Batch command for the organizers.** `run.py` takes a folder of audio, treats each file as a call (files named `X_1`, `X_2`… as turns of one call) and saves the reply text, reply audio and results.jsonl/csv with transcript, decision, tool calls and the document chunks used. A failing file is logged and the batch continues.
- **Tested with real audio.** `make_test_audio.py` voices practice questions and conversation PC002 with a second Gnani voice; `run.py` handled every file (`sample_outputs/`): the 4-turn call was verified from spoken digits and escalated, and a silent clip got a polite request to repeat. Gnani's firewall rejected 2 files with HTTP 403; the agent now re-encodes the audio (inaudible change) and retries, which fixed both.
- **Web demo.** `app.py` (Gradio): record or type, hear the reply, and see what STT heard, the decision, tool calls and the sources.

What went wrong and what I changed

Problem seen in the practice runs	Cause	Fix
Invented policies (Step 1, and early Step 2 on gaps)	Model fills gaps with plausible rules	Literal-grounding rule, worked 'don't know' example, search-before-don't-know check
Wrong refund dates (15–16 Oct instead of 17 Oct)	Model did date maths itself or ignored date_calc	All dates via date_calc; reply must contain the computed date
Asked for an order ID on general questions	Over-cautious model, worst on gpt-oss-20b	Needless-question check; rule-instead-of-question; refusal path
Required tool call sometimes skipped	Model forgot lookup_order on the turn the ID was given	Order lookup and verification moved into code
Missed Priority Resolution handover for a cracked laptop	Threshold section not retrieved	Damage/approval words pull in escalation sections; damage check
Crash: malformed tool-call JSON from the model	Final answer sent through a tool call	Final answer as plain DECISION / EVIDENCE / REPLY text; failed generations salvaged
Decision labels wrong although the reply was right (Step 3)	Weaker model mislabels its own reply	Label derived from the turn's actions (ticket, denial, question, verification)
Spoken '4533' read as 4453; STT 403 on some files	Cardinal numbers; firewall on audio bytes	Cardinal and paired-digit parser; re-encode and retry
'We don't offer store pickup' (invented); privacy refusal labelled answer	Model states a negative policy no document has; label missed denials of others' data	Unsupported-denial check sends it back; privacy request + denial labelled refuse
Free-tier daily limits and network drops	200K tokens/day per model on Groq's free plan	Results saved after every item, --resume / --redo, retries with backoff

What I would do next

- Use a paid or self-hosted LLM so every evaluation runs on the same strong model, and report variance over 3 runs.
- Stream audio (Gnani realtime STT) and start speaking the first sentence early to cut response latency.
- Learn the retrieval weights and synonym map from the practice questions instead of setting them by hand, and add a cross-encoder re-ranker for the top 10 sections.
- Grow the test set with my own recorded calls (noisy rooms, fast speakers, code-mixed Hindi) and track a voice-only score separately from the text score.

Bonus work

Verifier loop; deterministic verification and tool calls; Hindi and Hinglish replies; spoken-number normalizer; unclear-audio, misheard-number and STT-firewall recovery; test-audio generator and sample outputs; Gradio web demo; resumable evaluator with per-item results in `results/`; multi-provider LLM client (Groq, Gemini, Cerebras) with fallback.